INSTITUT D'ENSEIGNEMENT SUPÉRIEUR DE RUHENGERI
B.P. 155, Ruhengeri, Rwanda
T : +250 788 90 30 30, +250 788 90 30 32, W : www.ines.ac.rw, E : info@ines.ac.rw

Artificial Intelligence
SWE 3513

INDIVIDUAL ASSIGNMENT 2

Musanze Harvest Intelligence Service

An individual reproducible machine learning system and API

Assessment detail	Requirement
Assessment type	Individual practical assignment
Final product	Reproducible Python ML pipeline, saved models and FastAPI service
Total marks	100 marks
Issued	Friday, 16 October 2026
Deadline	Monday, 19 October 2026 at 5:00 PM CAT (Africa/Kigali)
Submission	One ZIP file through the INES e-learning platform

Assignment decision

Build a small but complete AI service for a Musanze potato cooperative. The service must generate an individual dataset from your registration number, clean it, train and evaluate three kinds of machine learning, save all evidence, and serve a validated prediction through FastAPI. The assessor must be able to reproduce your results from one command.

Individual verification code. Use AI-A2-LLLL, where LLLL is the last four digits of your registration number. Display it in the command-line output, API model information and report.

Deadline rule. The Moodle timestamp is authoritative. Submit before Monday, 19 October 2026 at 5:00 PM CAT.

Scenario and professional purpose

A potato cooperative in Musanze receives farm records in inconsistent spreadsheets. Its field officer needs an evidence-based tool that can estimate harvest quantity, identify farms at high production risk and group farms with similar conditions. Internet access may be unreliable, so model training and command-line prediction must work locally. A simple API is required for later integration with a mobile application.

Learning outcomes

Turn a real decision problem into regression, classification and clustering tasks.
Generate, inspect, clean and split a reproducible dataset without target leakage.
Build baselines, compare candidate models and report appropriate metrics.
Interpret model errors and clusters instead of presenting accuracy alone.
Save models and artifacts, validate prediction input and expose inference through FastAPI.
Test, document and defend an AI system as its individual author.

Individual data signature

Create data/generate_data.py. Let LLLL be the integer formed by the last four digits of your registration number. The same script must recreate the submitted CSV on a clean computer.

Item	Individual rule	Required evidence
Random seed	LLLL	Use numpy.random.default_rng(LLLL).
Base rows	240 + (LLLL mod 41)	Produces between 240 and 280 records before duplicates.
Verification code	AI-A2-LLLL	Print and store it in every major report.
CSV format	Four decimal places	Save numeric values with float_format="%.4f".
Dataset fingerprint	SHA-256	Hash the final CSV bytes and write artifacts/data_fingerprint.txt.

Required data generator

Column	Generation rule	Use
rainfall_mm	Uniform 600 to 1400	Numeric feature
avg_temp_c	Uniform 12 to 26	Numeric feature
soil_moisture_pct	Uniform 20 to 80	Numeric feature
fertilizer_kg	Uniform 0 to 250	Numeric feature
field_size_ha	Uniform 0.2 to 4.0	Numeric feature
season	Random A or B	Categorical feature
cooperative_support	Random 0 or 1	Binary feature

Column	Generation rule	Use
yield_tons	Formula below	Regression target
high_risk	Yield per hectare below its median	Classification target

- **Yield formula.** yield_tons = 0.004 × rainfall + 0.06 × soil moisture + 0.02 × fertilizer + 1.30 × field size − 0.25 × |temperature − 18| + 0.70 × cooperative support + season effect + noise. Use +0.50 for season A, −0.20 for season B and normal noise with mean 0 and standard deviation 0.80. Clip the final target at a minimum of 0.50 tons.
- **Deliberate data problems.** After creating the base rows, let m = round(0.05 × base row count). Use the same generator to choose m unique row positions for soil_moisture_pct and another m unique positions for fertilizer_kg, set those values to missing, then append three exact duplicate rows. Your pipeline must detect and handle these issues; do not manually edit the CSV.

Required system behavior

No.	Component	Minimum evidence
1	Reproducible entry point	run_all.py accepts --student-id and --seed, recreates data, cleans it, trains all tasks and regenerates every artifact.
2	Data validation	Reject a seed that is not LLLL. Check required columns, ranges, missing values, duplicates and target leakage.
3	Preprocessing	Use train-only fitting, numeric imputation and scaling where needed, categorical encoding and a fixed train/test split with random_state=LLLL.
4	Regression	Compare a mean baseline with at least two regression models. Report MAE, RMSE and R^2 on the held-out test set.
5	Classification	Compare a majority baseline with at least two classifiers. Report precision, recall, F1, confusion matrix and ROC-AUC when valid.
6	Clustering	Scale input features, test K from 2 to 6, use silhouette score to justify K and write a plain-language profile for each selected cluster.
7	Prediction CLI	predict.py loads saved artifacts and accepts one JSON record. It must return yield estimate, risk prediction, probability, cluster and verification code.
8	FastAPI service	Provide GET /health, GET /model-info and POST /predict. Use typed validation and return useful 4xx errors for invalid input.

No.	Component	Minimum evidence
9	Automated tests	Provide at least six pytest tests covering valid inference, missing field, wrong type, range error, reproducibility and one API response.
10	Responsible AI	Document intended use, limitations, synthetic-data status, risks, prohibited use and human-review requirement.

Modeling rules

Do not use yield_tons, yield per hectare or high_risk as input features for classification. This is target leakage.

Do not fit imputers, encoders, scalers or feature selection on the full dataset before the train/test split.

Use a Pipeline or ColumnTransformer so training and inference apply the same transformations.

Select a final model from measured validation or test evidence, not from an unsupported preference.

Set and record all random states. Re-running run_all.py twice must reproduce the same dataset fingerprint and metrics within normal floating-point tolerance.

The service is a classroom decision-support prototype. It must not claim to replace an agronomist or determine access to finance, insurance or public services.

Required artifacts

Path	Required content
artifacts/regression_metrics.json	Baseline and candidate MAE, RMSE, R^2; selected model and reason
artifacts/classification_metrics.json	Baseline and candidate precision, recall, F1, confusion matrix and ROC-AUC
artifacts/clustering_report.json	Scores for K=2…6, selected K, centers or profiles and interpretation
artifacts/test_predictions.csv	Row ID, actual values, predicted values, probabilities, errors and cluster
artifacts/plots/	residuals.png, confusion_matrix.png and clusters.png
models/	Saved preprocessing and final models loaded by predict.py and app.py

Technical report PDF

Submit a five to seven page PDF named AI_A2_REGDIGITS_REPORT.pdf. It must agree with the generated JSON and CSV artifacts and remain readable at 100 percent zoom.

Page	Required section	Assessment evidence
1	Identity and decision problem	Full name, registration number, verification code, stakeholder, prediction questions and ethical boundary
2	Data and reproducibility	Data dictionary, personal seed rule, row count, missing and duplicate evidence, SHA-256 fingerprint
3	Pipeline design	Train/test separation, preprocessing flow, leakage controls and model candidates
4	Regression and classification	Comparable metrics, residual and confusion-matrix interpretation, selected models and one important error
5	Clustering	K comparison, selected clusters, profiles and operational meaning
6	API and testing	Endpoint contract, valid and invalid examples, test summary and failure behavior
7 if used	Model card and reflection	Intended use, limitations, risks, human oversight and what you would improve with real data

Minimum command interface

```
python -m venv .venv
# activate the environment, then install dependencies
pip install -r requirements.txt
python run_all.py --student-id ug2424671@ines.ac.rw --seed 4671
python predict.py --json samples/valid.json
pytest -q
uvicorn app:app --host 127.0.0.1 --port 8000
```

Important. The values above are only an example. Replace the username and seed with your own registration details. requirements.txt must use compatible version ranges or exact versions that you tested.

Code and engineering expectations

Use Python, NumPy, Pandas, scikit-learn, Matplotlib or Seaborn, joblib, FastAPI and pytest. Additional packages must be justified in README.md.

Separate data generation, validation, preprocessing, modeling, evaluation and API logic into reusable modules under src/.

Use pathlib paths, clear function names, docstrings for public functions and helpful error messages.

Never include passwords, tokens, private keys, virtual environments, cache folders, notebook checkpoints or unrelated datasets.

A notebook may support exploration, but the assessed system must run from the required Python scripts without manually executing notebook cells.

Personalization and originality

Your dataset size, random sequence, verification code, metrics, plots and model behavior must follow your own registration digits. You must also write your own interpretation of one regression error, one classification error and one farm cluster. Submissions with mismatched seeds, copied fingerprints, identical explanations or unexplained model code may be referred for academic integrity review.

Marking rubric

Criterion	Marks	Full-credit evidence
Identity and reproducibility signature	8	Correct student identity, LLLL seed, verification code, row count and stable SHA-256 fingerprint.
Data generation and validation	12	Correct formula and distributions; deliberate missing values and duplicates; robust schema and range checks.
Preprocessing and leakage control	10	Train-only transformations, appropriate imputation and encoding, reusable pipeline and no target leakage.
Regression modeling	12	Valid baseline and two candidates; correct MAE, RMSE and R^2; justified selection and residual interpretation.
Classification modeling	15	Valid baseline and two candidates; precision, recall, F1, confusion matrix and ROC-AUC; error analysis and justified selection.
Clustering and interpretation	10	K=2…6 comparison, justified silhouette-based choice and useful farm profiles without using targets.
CLI and FastAPI inference	10	Saved artifacts load correctly; valid output; typed request; health, model-info and predict endpoints; safe 4xx errors.
Testing and code quality	8	At least six meaningful passing tests, modular source, deterministic behavior and documented dependencies.
Technical report and responsible AI	10	Complete evidence, matching metrics and plots, clear limitations, human oversight and honest interpretation.
Submission evidence and defense readiness	5	Correct package, concise video, commit hash, test log, disclosure and ability to explain or modify code.
Total	100	

Assessor verification sequence

The assessor may use the following sequence and a hidden input record. Be ready to explain and make one small change to your own code.

1. Confirm the student identity, AI-A2-LLLL code, final commit hash and dataset fingerprint.

2. Run run_all.py with the submitted student ID and seed, then compare the recreated fingerprint and artifact list.

3. Open the three metrics files and ask the student to explain one model choice, one error and one cluster.

4. Run predict.py with one valid hidden record and one invalid record.

5. Start FastAPI, check /health and /model-info, then submit valid and invalid requests to /predict.

6. Run pytest -q and inspect whether the tests exercise failure paths rather than only happy paths.

7. Ask one code question and request one small live modification, such as changing a range rule or adding a response field.

Submission package

Submit one ZIP file named AI_A2_REGDIGITS.zip. REGDIGITS is the registration number without the slash. The extracted folder must use the same name. Example: registration number 24/24671 uses AI_A2_2424671.zip.

AI_A2_REGDIGITS/

README.md
requirements.txt
run_all.py
predict.py
app.py
src/ reusable Python modules
data/generate_data.py
data/AI_A2_REGDIGITS.csv
models/ saved preprocessing and models
artifacts/ JSON, CSV, TXT and PNG outputs
samples/valid.json
samples/invalid.json
tests/
docs/AI_A2_REGDIGITS_REPORT.pdf
evidence/AI_A2_REGDIGITS_DEMO.mp4
evidence/AI_A2_REGDIGITS_TEST_LOG.pdf
evidence/AI_A2_REGDIGITS_DECLARATION.pdf
evidence/AI_USE.md

README requirements

Start with the student's full name, registration number, institutional username and AI-A2-LLLL code. Include the problem, exact setup and run commands, Python version, tested operating system, artifact list, API examples, repository URL, final commit hash, known limitations and a clear statement that the dataset is synthetic.

Required demonstration video

Submit one continuous, unedited MP4 of no more than two minutes. Voice narration must identify the student. The terminal clock must be visible at the beginning or end.

1. Show a handwritten card with full name, registration number and AI-A2-LLLL code, then show the final commit hash.

2. Run run_all.py and show the recreated fingerprint and generated artifact list.

3. Open one metrics JSON and one plot, then explain one result without reading a prepared script.

4. Run predict.py with one valid record and one deliberately invalid record.

5. Open FastAPI documentation, call /health and /predict, and show one validation error.

6. Run pytest -q and end by showing the report, declaration and terminal clock.